

SUMMARY

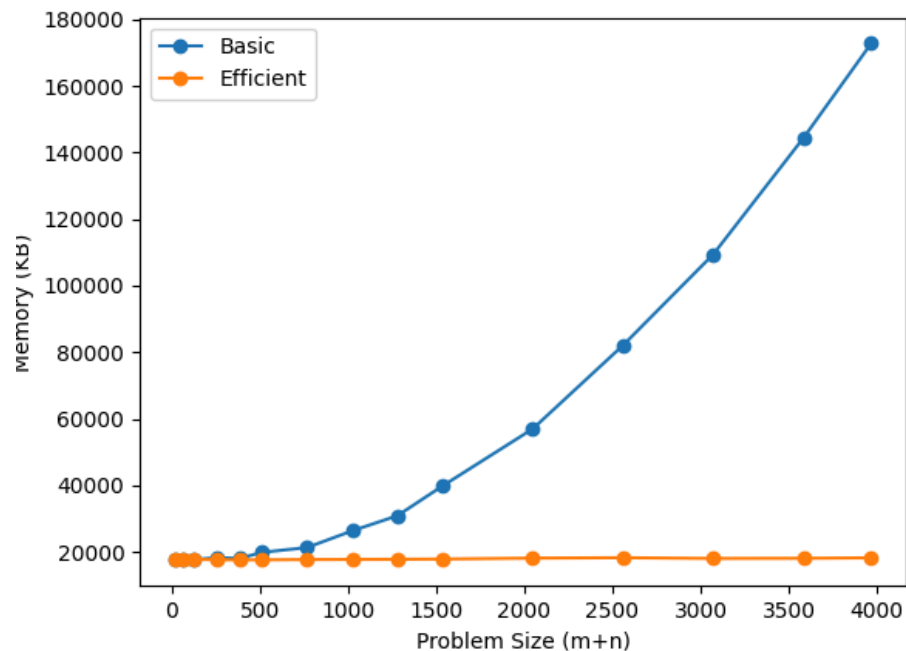
USC ID/s: 9541653447, 1711548333, 5452225948, 5179148878, 4638603957

Datapoints

M+N	Time in MS (Basic)	Time in MS (Efficient)	Memory in KB (Basic)	Memory in KB (Efficient)
16	0.729	0.9101	17776.0	17572.0
64	1.0396	1.4361	17792.0	17728.0
128	1.922	4.6087	17704.0	17768.0
256	5.3267	10.2737	18280.0	17656.0
384	15.5663	23.3791	18056.0	17592.0
512	37.0265	55.8369	19864.0	17612.0
768	54.9868	95.468	21312.0	17704.0
1024	111.2085	190.9652	26332.0	17732.0
1280	146.8276	332.3073	30864.0	17772.0
1536	233.1042	472.0184	39804.0	17844.0
2048	459.4956	889.4488	56980.0	18136.0
2560	694.0018	1353.1042	82080.0	18260.0
3072	974.8902	1910.1341	109420.0	18032.0
3584	1339.7426	2334.1929	144444.0	18084.0
3968	1539.6981	3052.9366	172816.0	18228.0

Insights

Graph1 – Memory vs Problem Size (M+N)



Nature of the Graph (Logarithmic/ Linear/ Polynomial/ Exponential)

Basic: Polynomial (Quadratic)

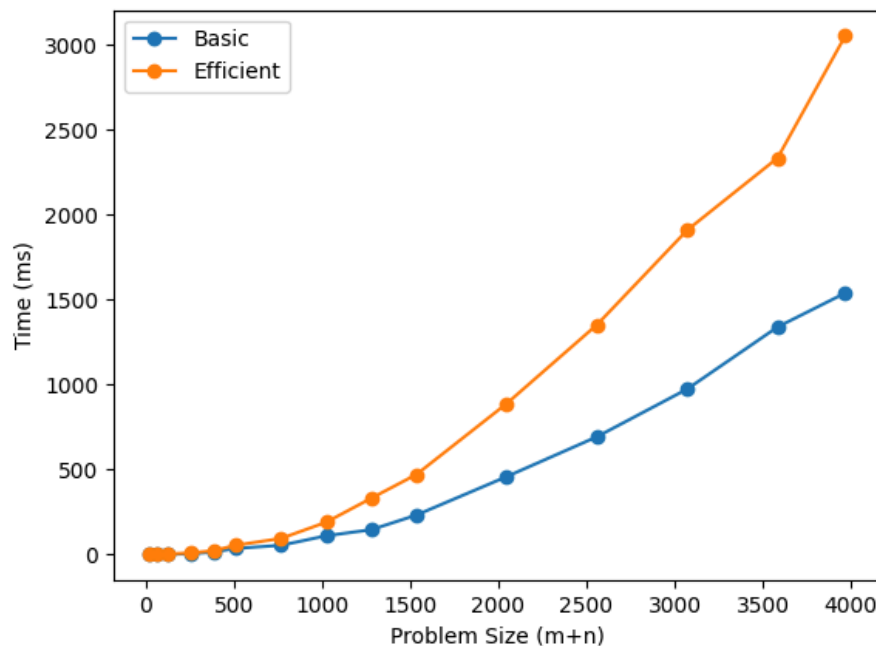
Efficient: Linear

Explanation: The memory usage of the basic algorithm increases rapidly as the problem size grows. This is because it constructs a full dynamic programming table of size $m * n$, resulting in quadratic space complexity. This is reflected in the graph as a steep upward curve.

In contrast, the efficient algorithm maintains only a small portion of the DP table at any time using a divide-and-conquer approach. As a result, its memory usage grows linearly with input size $O(m+n)$, but in practice appears almost constant (around 17,000–18,000 KB in the graph).

The graph clearly demonstrates a significant difference in memory consumption between the two approaches. While the basic algorithm becomes increasingly memory-intensive for larger inputs, the efficient algorithm remains stable and scalable. This highlights the advantage of the memory-efficient algorithm for large-scale problems where memory is a limiting factor.

Graph2 – Time vs Problem Size (M+N)



Nature of the Graph (Logarithmic/ Linear/ Polynomial/ Exponential)

Basic: Polynomial (Quadratic)

Efficient: Polynomial (Quadratic)

Explanation: The running time of both the basic and memory-efficient algorithms increases as the problem size grows. This is because both approaches solve the sequence alignment problem using dynamic programming, resulting in quadratic time complexity $O(mn)$.

The graph shows that the efficient algorithm consistently takes more time than the basic algorithm. This is due to the additional overhead introduced by the divide-and-conquer approach, including recursive calls and repeated computations.

In contrast, the basic algorithm computes the solution in a single pass, making it faster in practice despite its higher memory usage.

Overall, the graph highlights a trade-off between time and memory: the basic algorithm is faster but memory-intensive, while the efficient algorithm is slower but more memory-efficient.

Trivial Problem Size

The trivial problem size is the threshold below which the basic algorithm is preferred over the memory-efficient algorithm. Based on the experimental results, the basic algorithm consistently achieves lower runtime than the efficient algorithm across all tested input sizes. This is because the efficient algorithm introduces additional overhead through recursion and repeated computations.

However, the basic algorithm requires significantly more memory, as it stores the entire dynamic programming table, leading to rapid growth in memory usage as the input size increases. In contrast, the efficient algorithm maintains low and nearly constant memory usage.

As a result, the optimal trivial size depends on system constraints. When sufficient memory is available, the basic algorithm is preferable due to its faster execution. For larger inputs or memory-limited environments, the efficient algorithm should be used to avoid excessive memory consumption.

Contribution

(Please mention what each member did if you think everyone in the group does not have an equal contribution, otherwise, write "Equal Contribution")

Equal Contribution